Claude Code — Built-in Tools Reference

> *Source: code.claude.com/docs/en/tools-reference*
> *Last verified: May 2026*

Claude Code ships with a set of built-in tools that it uses autonomously to understand and modify codebases. Tool names listed here are the exact strings used in permission rules, subagent tool lists, and hook matchers.

Permission Key

Symbol	Meaning
✅	No explicit permission required
🔐	Requires user permission / approval

Tools Table

Tool	Permission	What It Does
`Agent`	✅	Spawns a subagent with its own context window to handle a complex or isolated task autonomously
`AskUserQuestion`	✅	Presents multiple-choice questions to the user to gather requirements or resolve ambiguity before proceeding
`Bash`	🔐	Executes shell commands in a persistent bash session; used for running tests, git commands, package managers, build tools, and general terminal operations
`CronCreate`	✅	Schedules a recurring or one-shot prompt/task within the current session (tasks are cleared when Claude exits)
`CronDelete`	✅	Cancels a scheduled task by its ID
`CronList`	✅	Lists all scheduled tasks currently active in the session
`Edit`	🔐	Makes targeted, surgical edits to a specific file using exact string replacement — preferred over full rewrites for small changes
`EnterPlanMode`	✅	Switches Claude into plan mode so it designs an approach and gets approval before writing any code
`EnterWorktree`	✅	Creates an isolated git worktree and switches into it — useful for parallel Claude Code sessions on separate branches
`ExitPlanMode`	🔐	Presents the plan to the user for approval and exits plan mode to begin execution
`ExitWorktree`	✅	Exits the current worktree session and returns Claude to the original working directory

`Glob`	✅	Finds files by name/path pattern matching (e.g., `**/*.ts`); results sorted by modification time, capped at 100 files
`Grep`	✅	Searches file *contents* for text patterns using ripgrep syntax; can return file paths only, matching lines, or context around matches
`ListMcpResourcesTool`	✅	Lists resources (prompts, data, schemas) exposed by connected MCP servers
`LSP`	✅	Provides code intelligence via language servers — jump to definitions, find references, get type info, list symbols, trace call hierarchies, and catch type errors after edits
`NotebookEdit`	🔐	Modifies cells in Jupyter (`.ipynb`) notebooks — add, edit, delete, or reorder cells
`PowerShell`	🔐	Executes PowerShell commands natively on Windows (opt-in preview; set `CLAUDE_CODE_USE_POWERSHELL_TOOL=1` to enable)
`Read`	✅	Reads the contents of one or more files; core tool for understanding code before making changes
`ReadMcpResourceTool`	✅	Reads a specific MCP resource by its URI
`SendMessage`	✅	Sends a message to an agent team teammate or resumes a stopped subagent by its agent ID *(requires `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` or `--agent-teams` flag)*
`Skill`	🔐	Executes a pre-built skill (reusable prompt-based workflow) within the main conversation
`TaskCreate`	✅	Creates a new task in the session task list
`TaskGet`	✅	Retrieves full details for a specific task by ID
`TaskList`	✅	Lists all tasks with their current status (pending, in-progress, done, blocked)
`TaskOutput`	✅	*(Deprecated)* Retrieves output from a background task — prefer using `Read` on the task's output file path instead
`TaskStop`	✅	Kills a running background task by its ID
`TaskUpdate`	✅	Updates a task's status, dependencies, or details; can also delete tasks
`TeamCreate`	✅	Creates an agent team with multiple specialized teammates *(requires `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` or `--agent-teams` flag)*
`TeamDelete`	✅	Disbands an agent team and cleans up all teammate processes *(requires agent teams flag)*
`TodoWrite`	✅	Manages the session task checklist in non-interactive / headless mode; interactive sessions use `TaskCreate`, `TaskList`, `TaskGet`, and `TaskUpdate` instead

`ToolSearch`	✅	Searches for and loads deferred MCP tools on-demand when tool search is enabled — avoids stuffing all tool definitions into context upfront
`WebFetch`	🔐	Fetches content from a specified URL; prompts once per new domain in default mode; sets a `Claude-User` User-Agent header
`WebSearch`	🔐	Runs a web search via Anthropic's search backend and returns result titles and URLs (US only)
`Write`	🔐	Creates a new file or completely overwrites an existing one — use `Edit` for targeted changes

---

Noteworthy Behaviors

Bash

- Working directory **persists** across commands within a session
- Environment variables do **not** persist (re-export each command as needed)
- Default timeout: **2 minutes** per command (Claude can request up to 10 min)
- Default output cap: **30,000 characters** (raise with `BASH_MAX_OUTPUT_LENGTH`)
- Long-running processes (dev servers, watchers) can be run in the background with `run_in_background: true`

Edit vs Write

- Use **`Edit`** for targeted changes to existing files (exact string replacement)
- Use **`Write`** when creating new files or when a full rewrite is appropriate

Glob vs Grep

- **`Glob`** → finds files by *name/path pattern* (does not respect `.gitignore` by default)
- **`Grep`** → finds files by *content pattern* (respects `.gitignore` by default)

LSP

- Inactive until a code intelligence plugin is installed for the target language
- Automatically reports type errors and warnings after each file edit

WebFetch

- Redirects to a different host are **not** followed automatically — Claude fetches the redirect target in a second call

---

Extending Claude Code Tools

Method	What It Does
MCP Server	Connect an external server to add fully custom tools
Skill	Write a reusable prompt-based workflow; invoked via the existing `Skill` tool
Permissions	Use `/permissions` or `settings.json` to allow/deny specific tools

Check Available Tools in a Session

```
What tools do you have access to?
```

Ask Claude directly for a conversational summary. For exact MCP tool names, run `/mcp`.